



<Bolsa Futuro Digital>

<aula 20>

Turma de FrontEnd – IF UFRGS

● Variáveis



Uma variável pode receber qualquer tipo de valor, número ou texto, mas sempre um **ÚNICO** valor por vez. Podemos usar o concole para mostrar o valor das variáveis.

```
// let - cria a variável
// = - significa recebe
// Cria a variável nome que recebe 'Gabriel'
let nome = 'Gabriel'

// Cria a variável idade que recebe 25
let idade = 25

// Exibe nome e idade no console
console.log(nome, idade)
```

● Variáveis compostas



Até agora uma variável guardava **apenas um valor** por vez.

Uma **variável composta** (também chamada de **array** ou **vetor**) consegue guardar **vários valores** em uma única estrutura, como uma lista.

Pense em um **gaveteiro**: cada gaveta guarda um valor diferente, mas todas fazem parte do mesmo móvel.

variável simples

```
let nome = 'Gabriel'
```

VS

variável composta (array)

```
let nome = ['Bia', 'Ana', 'Carlos']
```

● Variáveis compostas



Para criar um array usamos `let`, um nome e os valores entre colchetes `[]` separados por vírgula.

```
// Array de strings (texto)
let frutas = ['maçã', 'banana', 'laranja']

// Array de numbers (números)
let notas = [8.5, 7.0, 9.2, 6.8]

// Array misto (vários tipos)
let dados = ['João', 20, true]
```

● Índices do array



Cada valor do array tem um **índice**, que indica a sua posição. Em JavaScript a contagem **começa sempre no zero!**

'maçã'	'banana'	'laranja'	'uva'
[0]	[1]	[2]	[3]

```
let frutas = ['maçã' , 'banana', 'laranja', 'uva']
```

```
// Exibe o elemento na posição [0] (1º)
```

```
console.log(frutas[0]) // maçã
```

```
// Exibe o elemento na posição [2] (3º)
```

```
console.log(frutas[2]) // laranja
```

● Inserindo elementos



O método **push()** adiciona um novo elemento **ao final** do array.

É como acrescentar mais um item no final da sua lista de compras – os outros itens não se movem!

```
let frutas = ['maçã', 'banana']

// Adiciona 'laranja' ao final
frutas[2] = 'laranja'

// Irá exibir ['maçã', 'banana', 'laranja']
console.log(frutas)

// Adiciona 'uva' ao final
frutas.push('uva')

// Irá exibir ['maçã', 'banana', 'laranja', 'uva']
console.log(frutas)
```

● Tamanho do array



Podemos descobrir o tamanho de um array utilizando o método `length`, ele diz quantos elementos existem.

```
// .length mostra o comprimento ou quantos
// elementos existem dentro de um array
let frutas = ['maçã', 'banana', 'laranja']

// o console exibe 3 elementos
console.log(frutas.length)

frutas.push('uva')

// agora o console exibe 4 elementos
console.log(frutas.length)
```

● Ordenando elementos



Podemos utilizar o método `sort()` para ordenar os elementos de um array. Strings serão colocadas em ordem alfabética e Numbers em ordem crescente.

```
let nomes = ['Gabriel', 'Ana', 'Carlos']

// coloca em ordem alfabética
nomes.sort()
console.log(nomes)
// ['Ana', 'Carlos', 'Gabriel']

let numeros = [2, 5, 1, 9, 8]

// coloca em ordem crescente
numeros.sort()
console.log(numeros)
// [1, 2, 5, 8, 9]
```

● Ordenando elementos



Podemos utilizar o método `sort()` em arrays com valores mistos, ele funciona da mesma maneira mas agrupa os tipos na seguinte ordem: Number → String → Boolean

```
// Podemos ordenar arrays com valores mistos
let misto = [25, true, 'Gabriel', 1, false, 'Ana']

// ordena em: Number -> String -> Boolean
misto.sort()

console.log(misto)
// [1, 25, 'Ana', 'Gabriel', false, true]
```

● Percorrendo um array



Podemos percorrer um array usando o `for`. Definimos o índice do vetor como o contador do nosso loop, ele inicia em zero e vai até `array.length`

```
let frutas = ['maçã', 'banana', 'laranja']

// Exibindo o primeiro elemento
console.log(frutas[0])

// Exibindo todos os elementos do array
console.log(`Na posição 0 temos o elemento ${frutas[0]}`)
console.log(`Na posição 1 temos o elemento ${frutas[1]}`)
console.log(`Na posição 2 temos o elemento ${frutas[2]}`)

// Usando for para percorrer o array
for( let i=0; i < frutas.length; i++) {
  console.log(`Na posição ${i} temos o elemento ${frutas[i]}`)
}
```

● Percorrendo um array



Usamos tanto o `for` para percorrer um array que existe um atalho para isso, o `for...in`. Definimos uma variável como índice do array dentro do `for` e ele automaticamente faz o loop.

```
// Usando for para percorrer o array
for( let i=0; i < frutas.length; i++) {
|   console.log(`Na posição ${i} temos o elemento ${frutas[i]}`)
}

// Usando for...in para percorrer o array
for(let i in frutas) {
|   console.log(`Na posição ${i} temos o elemento ${frutas[i]}`)
}
```

● Buscando valores



O método `indexOf()` busca um valor no array e retorna sua posição. Se o valor não for encontrado, ele retorna `-1`.
Útil para verificar se um item já existe na lista!

```
let frutas = ['maçã', 'banana', 'laranja']  
  
// Banana está no array e tem index [1]  
console.log(frutas.indexOf('banana'))  
  
// Uva não está no array, portanto seu index é [-1]  
console.log(frutas.indexOf('uva'))
```

● Buscando valores



```
// Criamos um array
let num = [5, 8, 2, 9, 3]

// Escolhemos a posição de um elemento
let pos = num.indexOf(8)

// Verifica se o elemento está no array
if (pos == -1) {
  // Se a posição for -1 ele não está no array
  console.log('O valor não foi encontrado!')
} else {
  // Se não ele está na posição
  console.log(`O valor está na posição ${pos}`)
}
```

● Funções

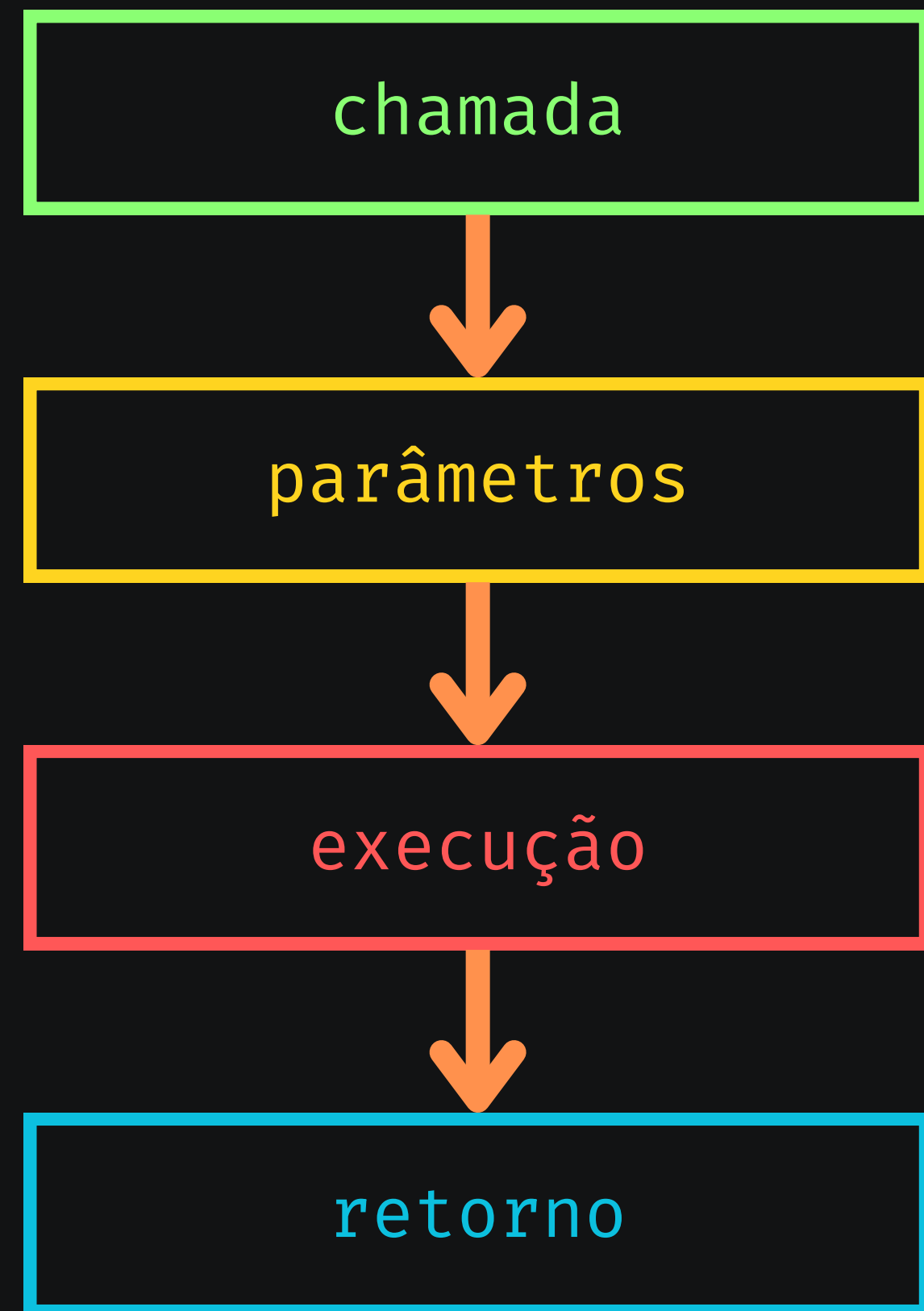


Uma **função** é uma rotina – um bloco de código que pode ser **reutilizado** várias vezes e só é executado quando é **chamado**.

Pense em uma **receita de bolo**: você escreve os passos uma vez e pode fazê-la sempre que quiser, trocando apenas os ingredientes.

Uma função tem **4 partes**:

- **chamada**: quando você a executa;
- **parâmetros**: os dados de entrada;
- **ação**: o processamento;
- **retorno**: o resultado final.



● Funções

Usamos a palavra **function** para criar uma função, damos um **nome** e colocamos o bloco de **código** entre chaves **{}**

Tudo que estiver entre as chaves **{}** só será executado quando a função for chamada.

Nem toda função precisa receber parâmetros e nem toda função precisa retornar algo.

```
// Cria a função soma com
// os parâmetros n1 e n2
function soma(n1, n2) {

    // Executa a soma
    let s = n1 + n2

    // Retorna o resultado
    return s
}

// O código inicia aqui
let a = 5
let b = 7

// A função é chamada
let s = soma(a, b)
```

● Funções



Em JavaScript, funções são valores como qualquer outro. Assim podemos atribuir uma função a uma variável.

Essa forma é chamada de function expression e funciona exatamente igual a uma função normal.

```
// A variável recebe uma função
let dobro = function(n) {
  return n*2
}

console.log(dobro(2)) // 4
console.log(dobro(4)) // 8
```

● Funções recursivas



Uma função **recursiva** é uma função que **chama a si mesma**.

É como colocar dois espelhos um de frente para o outro: a imagem se repete sem parar.

Para não repetir para sempre, toda função recursiva precisa de uma **condição de parada**.

```
// 5! = 5 x 4!  
// n! = n x (n-1)!  
  
function fatorial(n) {  
    if(n==1) {  
        // Se n for 1 retorna 1  
        return 1  
    } else {  
        // Se não  
        // multiplica pelo fatorial de n-1  
        return n * fatorial(n-1)  
    }  
}
```

● Exercícios



Exercício 1: Faça um array com o 5 nomes e faça um loop para exhibi-los em ordem alfabética, depois adicione mais nomes e exiba novamente em ordem.

Exercício 2: Crie um array com 5 notas de 0 a 10. Some todas elas e calcule a média, ao final mostre se o aluno foi aprovado ou reprovado (média > 6).

Exercício 3: Crie uma função que receba dois números e retorne qual deles é o maior ou se ambos são iguais.



</aula 20>